

EXP20: Prolog Program for a Planets Database

Name: Y. Roshitha (192424400)

Aim:

To develop a Prolog program that creates a database of planets with their position from the Sun and type, and to query it for specific information.

Algorithm:

- Step 1: Start the program.
- Step 2: Create a database containing planet names, positions, and planet types.
- Step 3: Store the information as Prolog facts.
- Step 4: Define a procedure to display all planet records.
- Step 5: Define a procedure to search for a planet using its position from the Sun.
- Step 6: Load the program into SWI-Prolog.
- Step 7: Execute the desired query.
- Step 8: Retrieve the matching planet information.
- Step 9: Display the result and terminate the program.

Source Code:

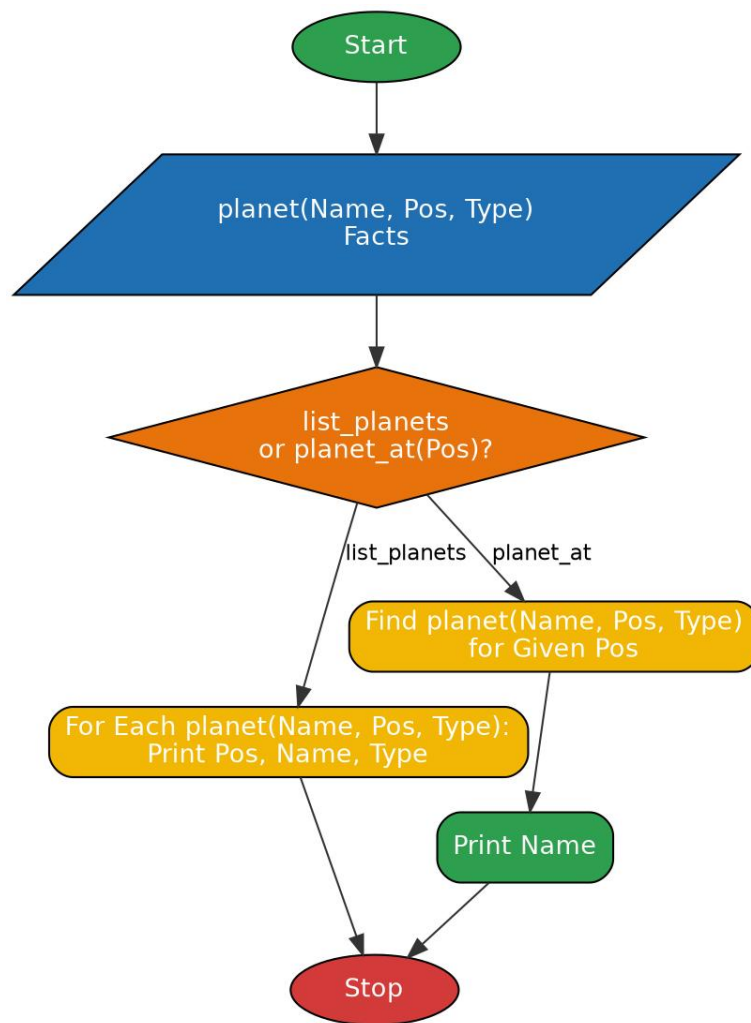
```
% Program for a Planets Database

planet(mercury, 1, terrestrial).
planet(venus, 2, terrestrial).
planet(earth, 3, terrestrial).
planet(mars, 4, terrestrial).
planet(jupiter, 5, gas_giant).
planet(saturn, 6, gas_giant).
planet(uranus, 7, ice_giant).
planet(neptune, 8, ice_giant).

list_planets :-
    forall(planet(Name, Pos, Type),
        format("~w. ~w (~w)~n", [Pos, Name, Type])).

planet_at(Pos) :-
    planet(Name, Pos, _),
    format("~w~n", [Name]).
```

Flowchart:



Output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- cd('C:/Users/yanar/Documents/CSA1717').
true.

?- [planet].
true.

?- list_planets.
1. mercury (terrestrial)
2. venus (terrestrial)
3. earth (terrestrial)
4. mars (terrestrial)
5. jupiter (gas_giant)
6. saturn (gas_giant)
7. uranus (ice_giant)
8. neptune (ice_giant)
true.

?- planet_at(3).
earth
true.

?- planet_at(5).
jupiter
true.

?-
```

Result:

Thus the Prolog program for the Planets database was written, executed successfully and the output was verified.